# Семинар 6

**Цель работы:** разработка очереди отложенных вызовов функций и методов классов на языке C++ с использованием шаблонов, полиморфизма и современных средств стандартной библиотеки для предметной области «Музей».

## 2. Описание предметной области

Предметной областью работы является ранее разработанная музейная система варианта 23, включающая музей, залы, коллекции экспонатов, гидов, посетителей и каталог музейных объектов. В рамках шестого семинара система была расширена механизмом очереди отложенных вызовов, позволяющим откладывать выполнение операций до момента их явного запуска.

### 2.1 Основные сущности системы

- Museum — основной объект музейной системы;
- Hall — залы музея;
- MuseumItem / Painting / Sculpture / Artifact — музейные экспонаты;
- Guide — экскурсовод;
- Visitor — посетитель;
- Catalog — каталог поиска объектов;
- MyContainer — пользовательский контейнер.

## 3. Архитектура системы

Система построена на основе шаблонов и полиморфизма. Для реализации очереди отложенных вызовов используются классы IPending, Pending<Callable, Args...> и PendingQueue. Класс IPending представляет собой базовый интерфейс отложенного вызова, Pending<Callable, Args...> реализует

шаблон хранения callable-объекта вместе с его аргументами, а PendingQueue обеспечивает постановку задач в очередь и их последующее выполнение. В архитектуре применяются variadic templates, std::tuple, std::apply и std::unique_ptr, что обеспечивает универсальность системы, поддержку произвольного количества аргументов и безопасное управление памятью.

## 4. Реализация системы

Все задачи хранятся через std::unique_ptr<IPending>

- enqueue() — добавление задачи;
- run() — выполнение одной задачи;
- runAll() — выполнение всей очереди;
- empty() — проверка пустоты;
- size() — определение размера очереди.

## 5. Интеграция с музейной системой

Очередь отложенных вызовов применяется для выполнения операций музейной системы:

- добавление новых залов;
- добавление экспонатов;
- формирование маршрутов гидов;
- назначение гидов посетителям;
- вывод информации о музее;
- выполнение экскурсионных сценариев.

## 6. Демонстрация работы программы

В ходе выполнения программы последовательно демонстрируются:

- создание очереди PendingQueue;
- добавление функций и методов через enqueue();

– подтверждение отсутствия немедленного выполнения;

– выполнение отдельных задач через run();

– выполнение всех задач через runAll();

– проверка изменения состояния музейной системы;

```
[2] Создана PendingQueue
  empty(): да, size(): 0
  Используются вариадические шаблоны: enqueue принимает
  разные callable и разное число аргументов.

[3] Добавляем отложенные вызовы через enqueue
  - свободная функция: printDeferredMessage(std::string)
  - методы Museum: addHall, addItem, showInfo
  - методы Guide: addToRoute
  - методы Visitor: setGuide, askGuideName
  После enqueue: empty(): нет, size(): 9
  Предметная область ещё не изменилась:
  Залов: 0, экспонатов: 0, остановок в маршруте: 0
```

Рисунок 1 – Создание очереди и постановка задач через enqueue().

```
[4] Выполняем первые вызовы через queue.run()
Отложенное сообщение: свободная функция выполнена
  После run() 1: size() = 8
  После run() 2: size() = 7
  После run() 3: size() = 6
  Промежуточное состояние:
  Залов: 1, экспонатов: 1, остановок в маршруте: 0
```

Рисунок 2 – Выполнение отдельных задач методом run().

```
[5] Выполняем оставшиеся вызовы через queue.runAll()
Посетитель Ольга общается с гидом Мария
Музей: Учебный музей отложенных вызовов
Количество залов: 1
Количество экспонатов: 2
  После runAll(): empty() = да, size() = 0
```

Рисунок 3 – Полное выполнение очереди методом runAll().

```
[6] Итоговое состояние:
  Залов: 1, экспонатов: 2, остановок в маршруте: 2
  Проверка Visitor::askGuideName():
Посетитель Ольга общается с гидом Мария
```

Рисунок 4 – Итоговое состояние музейной системы.

## 7. Вывод

В ходе выполнения работы была разработана универсальная очередь отложенных вызовов и успешно интегрирована в существующую музейную архитектуру. В результате был реализован интерфейс IPending, разработан шаблон Pending<Callable, Args...>, создан класс PendingQueue, обеспечена работа как со свободными функциями, так и с методами классов. В системе были использованы variadic templates, а также применены std::tuple и std::apply для хранения и выполнения вызовов. Разработанная система подтвердила эффективность применения шаблонов, полиморфизма.